

VOXFLOW — SPEECH FLUENCY & DISFLUENCY SESSION ANALYZER

FINAL PROJECT REPORT

Technical Answers for Real World Problems (TARP) — CSE1905

Submitted by:

23MIA1012 — Rayini Amarender Reddy

23MIA1111 — Peta Siva Nandhan Reddy

23MIA1140 — NayanUjwal Sri Tej

Faculty Supervisor:

Prof. GANALA SANTHOSHI

Department of Computer Science and Engineering in Business Analytics

School of Computer Science and Engineering (SCOPE)

Vellore Institute of Technology

Project Repository: <https://github.com/PetaSivaNandhanReddy/voxflow>

ABSTRACT

Speech disfluencies, such as repetitions, prolongations, and blocks, interrupt the smooth, natural rhythm of verbal expression. In speech therapy, tracking progress over time requires therapists to listen to recorded sessions and manually tally disfluent events by hand. This manual counting is time-consuming, mentally demanding, and susceptible to inter-rater variability. While self-supervised speech representations have recently advanced automated stuttering detection, existing research focuses almost exclusively on classifying isolated 3-second audio clips without addressing the end-to-end engineering challenges of dedicated hardware capture, error-checked serial transport, continuous multi-window session inference, temporal event consolidation, and persistent historical tracking.

VoxFlow is an end-to-end speech fluency session analyzer designed to bridge this research gap. The system consists of an embedded hardware capture unit built on an ESP32 microcontroller and an INMP441 MEMS digital microphone recording 16 kHz, 16-bit mono PCM audio. Audio frames are streamed over USB-UART using a custom-engineered framed binary protocol (VXF1) equipped with 32-bit sequence tracking and CRC32 payload verification. A Python serial bridge reconstructs session WAV files and dispatches them to a Flask REST backend. The backend executes multi-label disfluency classification using HuBERT-D, a fine-tuned HuBERT Transformer evaluated on speaker-exclusive partitions of SEP-28k and FluencyBank (519 unique speakers). Temporal event aggregation consolidates overlapping 3-second window inferences into distinct, non-redundant detected disfluency events. Session results, window probabilities, and aggregate metrics are stored in a local SQLite database and visualized via an interactive Streamlit dashboard. HuBERT-D achieves a test macro F1 of 0.4599 and a mean ROC-AUC of 0.8099 on unseen speakers, outperforming wav2vec 2.0 (0.4498) and an MFCC + SVM baseline (0.3121). All 38 automated unit and integration tests pass successfully, demonstrating a robust, fully verifiable session analysis platform. VoxFlow is a research prototype and is not a medical diagnostic system.

1. INTRODUCTION AND PROJECT OVERVIEW

1.1 Background

Stuttering is a neurodevelopmental speech condition characterized by frequent disruptions in the forward flow of verbal expression. Disfluent speech patterns are commonly categorized into three core behaviors: repetitions (involuntary iterations of sounds, syllables, or single-syllable words), prolongations (abnormal lengthening of continuous phonetic segments), and blocks (silent or audible postural fixations where airflow and vocalization are temporarily arrested). Stuttering affects approximately 1% of the global adult population and up to 5% of young children, impacting social communication, academic engagement, and emotional well-being.

Speech-language pathologists (SLPs) track progress across multi-week interventions by evaluating disfluency frequency and severity during structured speech sessions. In standard practice, therapists listen to audio or video recordings and tally individual disfluency occurrences to compute metrics such as the Percentage of Stuttered Syllables (%SS) and Stuttering Severity Instrument (SSI) scores. However, manual perceptual evaluation is labor-intensive, exhausting, and prone to subjective disagreement across raters [1]. Recent advances in computational speech processing have introduced machine learning models for automated disfluency detection using publicly available corpora such as SEP-28k [2] and FluencyBank [3]. Nevertheless, the vast majority of existing academic literature remains confined to classifying isolated 3-second audio clips, leaving an unaddressed engineering gap between algorithmic clip classification and continuous, hardware-integrated session analysis.

1.2 Project Overview

VoxFlow addresses this challenge by providing a complete, automated speech fluency and disfluency session analyzer. The system operates through a structured end-to-end pipeline:

- **Record session:** The user initiates recording using a dedicated ESP32 + INMP441 physical capture unit (3 to 60 seconds).
- **Process audio:** Audio is streamed over serial via the verified VXF1 protocol, reconstructed into a standardized 16 kHz mono WAV file, validated, and sliced into 3.0-second sliding analysis windows with a 1.0-second hop.

- **Detect disfluencies:** HuBERT-D performs multi-label inference on each window, generating independent probabilities for repetition, prolongation, and block classes.
- **Aggregate events:** A temporal event aggregation algorithm merges contiguous and proximate window activations (within a 1.5-second merge gap) into unified, non-redundant detected disfluency events.
- **Store results:** Session metadata, per-window probability vectors, and aggregated disfluency events are persisted to a relational SQLite database.
- **Review history:** An interactive Streamlit visual analytics dashboard displays session timelines, per-class breakdown charts, and longitudinal progress trends across multiple sessions.

1.3 Project Objectives

The core objectives of the VoxFlow project are defined as follows:

- **Objective 1 (Hardware Capture & Transport):** Design and construct a low-cost, dedicated digital speech capture unit using an ESP32 microcontroller and INMP441 MEMS microphone, with an error-checked framed binary streaming protocol (VXF1) ensuring zero silent data loss.
- **Objective 2 (Continuous Speech Analysis Engine):** Implement a backend processing pipeline that validates session audio (3–60 s), handles arbitrary session lengths via sliding windows (3.0 s window, 1.0 s hop), and reliably classifies core disfluencies.
- **Objective 3 (Self-Supervised Disfluency Modeling):** Fine-tune a self-supervised transformer model (HuBERT-D) on speaker-exclusive partitions of SEP-28k and FluencyBank, rigorously evaluating performance against wav2vec 2.0 and traditional acoustic baselines.
- **Objective 4 (Temporal Consolidation & Visual Analytics):** Develop a temporal event aggregator to prevent multi-counting across overlapping analysis windows, backed by relational database persistence and an intuitive dashboard for session review and longitudinal trend analysis.

2. PROBLEM STATEMENT, RESEARCH GAP AND NOVELTY

2.1 Problem Statement

Tracking speech fluency over longitudinal therapeutic interventions requires repeated speech assessments. In current therapeutic workflows, clinicians and speech therapists must manually listen to recorded sessions and hand-count individual repetitions, prolongations, and blocks. This manual observation demands substantial time, imposes a heavy cognitive workload, and complicates objective comparison of patient progress across consecutive weeks.

When automated speech processing is introduced to assist this workflow, sliding analysis windows applied across continuous speech naturally produce overlapping detections: a single 2-second block or prolongation spanning across multiple consecutive windows is counted multiple times if left unmerged. Furthermore, therapists require structured session-level information—including total session duration, speaking time, total disfluency count, disfluency rate per minute, and chronological event timelines—rather than isolated clip labels. Therefore, the core engineering problem is: how can we capture speech sessions with low-cost hardware, detect repetitions, prolongations, and blocks across continuous speech without redundant window multi-counting, and store structured session results so that therapeutic progress can be objectively reviewed over time? VoxFlow is designed as an assistive research tool to provide structured measurements and does not replace professional clinical diagnosis.

2.2 Existing Limitations

A rigorous examination of existing academic literature and software tools reveals several recurring limitations:

- **Clip-Level Isolation:** Nearly all published disfluency detectors operate exclusively on isolated 3-second audio segments pre-trimmed from podcasts or clinical archives, failing to evaluate continuous conversational speech.
- **Multi-Counting in Sliding Windows:** Applying clip classifiers across continuous sessions using sliding windows causes individual disfluencies to trigger in several consecutive windows, falsely inflating event counts without temporal consolidation.

- **Overoptimistic Splitting Schemes:** Many studies utilize random train/test splits that leak speaker identities across sets, producing inflated benchmark scores that fail to generalize to unseen speakers.
- **Absence of Integrated Capture Hardware:** Existing studies focus on offline machine learning notebooks and rarely specify the hardware capture chain, microphone electrical interfacing, or transmission error handling.
- **Lack of Longitudinal Storage & Visualization:** Academic prototypes rarely incorporate persistent database storage or dashboard interfaces for tracking fluency trends across multiple sessions.

2.3 Research Gap

In the set of published studies reviewed (detailed in Chapter 3), we did not find a unified system that integrates: (a) dedicated low-cost embedded speech capture with hardware-level transport verification, (b) continuous multi-window self-supervised disfluency detection evaluated under speaker-exclusive constraints, (c) temporal event aggregation to eliminate overlapping window redundancy, and (d) relational session persistence with longitudinal visual analytics. Existing work addresses isolated sub-problems—such as model architectures or dataset curation—leaving an end-to-end systems gap for session-level fluency tracking.

2.4 Novelty and Proposed Contribution

VoxFlow does not claim to introduce a novel neural architecture from scratch; rather, its novelty and contribution reside in its comprehensive, verified system-level integration:

- **Framed Hardware Streaming (VXF1):** Engineered an embedded binary protocol on ESP32 that embeds frame magic headers, 32-bit sequence numbers, payload lengths, and CRC32 checksums, ensuring transmission integrity across USB-UART.
- **Speaker-Exclusive HuBERT-D Modeling:** Fine-tuned HuBERT-D on 30,999 multi-annotator audio clips across 519 strictly partitioned, speaker-exclusive individuals, ensuring realistic generalization on unseen voices.
- **Temporal Event Aggregation Algorithm:** Formulated an algorithmic consolidation mechanism that clusters overlapping window detections within a 1.5-second gap into single, bounded detected disfluency events.

- **Complete Dual-Mode Software Pipeline:** Developed an integrated platform supporting both live hardware recording and standalone WAV file uploads, with relational SQLite persistence and Streamlit visual analytics.

3. LITERATURE REVIEW

3.1 Reviewed Literature

To establish a rigorous theoretical and empirical foundation for VoxFlow, we conducted a systematic literature survey of 13 key peer-reviewed research papers spanning speech disfluency datasets, acoustic modeling, deep learning architectures, and self-supervised speech representations.

3.1.1 Sheikh et al. (2022) [1]

Problem: Review machine learning techniques for stuttering identification, covering features, classifiers, datasets, and practical challenges. **Method:** Systematic survey of acoustic features (MFCCs, spectrograms), classical classifiers (SVMs, GMMs), and deep neural architectures. **Key finding:** Identified dataset fragmentation, lack of standardized evaluation protocols, and dominance of short-clip classification over continuous sessions. **Limitation / gap:** Survey paper; did not construct an end-to-end hardware capture, session aggregation, or clinical software tool. **Relevance to VoxFlow:** Validates VoxFlow's system-level objective to transition from isolated clip detection to full session tracking.

3.1.2 Lea et al. (2021) [2]

Problem: Scarcity of large-scale, naturalistic, publicly available audio datasets for multi-class stuttering event detection. **Method:** Introduced the SEP-28k dataset (28,177 3-second clips from public podcasts) annotated for repetitions, prolongations, blocks, interjections, and word repetitions. **Key finding:** Demonstrated multi-label disfluency detection in conversational audio is challenging, establishing baseline classifier benchmarks. **Limitation / gap:** Dataset audio is segmented into isolated 3-second clips without continuous temporal context or hardware streaming. **Relevance to VoxFlow:** SEP-28k serves as the primary training and evaluation corpus for VoxFlow's HuBERT-D classifier.

3.1.3 Bernstein Ratner and MacWhinney (2018) [3]

Problem: Absence of shared, open-access clinical archives and standardized transcripts for stuttering and speech fluency research. **Method:** Established FluencyBank under the TalkBank framework, providing standardized audio recordings and clinical transcripts. **Key finding:** Created an international open-access resource that enabled reproducible computational and clinical speech research. **Limitation / gap:** Archival database; does not provide automated machine learning detection models or hardware capture utilities. **Relevance to VoxFlow:**

FluencyBank provides the secondary benchmark corpus incorporated into VoxFlow's multi-annotator training dataset.

3.1.4 Howell et al. (2009) [4]

Problem: Need for accessible clinical recordings to analyze developmental stuttering trajectories across age cohorts. **Method:** Developed the University College London Archive of Stuttered Speech (UCLASS), offering recordings and orthographic transcriptions. **Key finding:** Showed acoustic differences in disfluency manifestations across age groups, establishing an early public benchmark. **Limitation / gap:** Smaller dataset recorded under legacy acoustic conditions; evaluated on isolated speech tokens without modern neural models. **Relevance to VoxFlow:** Provides foundational clinical taxonomy for defining repetition, prolongation, and block categories.

3.1.5 Kourkounakis et al. (2021) [5]

Problem: Detecting multi-class stuttered disfluencies directly from spectrogram representations without manual feature engineering. **Method:** Proposed FluentNet, integrating Squeeze-and-Excitation ResNet, BiLSTM layers, and attention mechanisms on log-mel spectrograms. **Key finding:** Achieved 91.75% accuracy on UCLASS and 86.7% on LibriStutter, proving attention mechanisms capture disfluency boundaries. **Limitation / gap:** Evaluated strictly on pre-cut audio segments; did not address streaming capture, frame transport, or session aggregation. **Relevance to VoxFlow:** Confirms the importance of capturing temporal dependencies in disfluency classification, inspiring VoxFlow's windowing design.

3.1.6 Sheikh et al. (2021) [6]

Problem: High computational complexity and inference latency of recurrent neural networks in multi-class stuttering classification. **Method:** Developed StutterNet, a Time-Delay Neural Network (TDNN) architecture trained on 2D MFCC acoustic features across context frames. **Key finding:** Outperformed baseline residual architectures while significantly lowering parameter counts through sub-sampled convolutions. **Limitation / gap:** Operates on isolated clips with handcrafted MFCC features, which have lower representational capacity than self-supervised embeddings. **Relevance to VoxFlow:** Demonstrates the feasibility of low-latency frame-level disfluency scoring and provides a baseline comparison.

3.1.7 Sheikh et al. (2023) [7]

Problem: Severe class imbalance across disfluency types and poor generalization across unseen speaker distributions in SEP-28k. **Method:** Proposed multi-contextual deep learning using SpecAugment data augmentation, class-balanced focal loss, and multi-branch features. **Key finding:** Class-balanced loss functions and data augmentation significantly improved detection of minority disfluencies (blocks and prolongations). **Limitation / gap:** Clip-level evaluation without

physical hardware integration, serial error checking, or session-level event consolidation. **Relevance to VoxFlow:** Directly motivated VoxFlow's adoption of class-weighted loss and disfluency-specific detection thresholds.

3.1.8 Bayerl et al. (2022a) [8]

Problem: Scarcity of clinical therapy datasets capturing deliberate fluency-shaping techniques alongside natural disfluencies. **Method:** Constructed the Kassel State of Fluency (KSoF) dataset, consisting of German speech recordings annotated by professional speech therapists. **Key finding:** Showed that clinical evaluation requires fine-grained disfluency categorization rather than binary fluent/disfluent discrimination. **Limitation / gap:** Language-specific (German) corpus focused on therapy modifications; evaluated on isolated clips without an integrated pipeline. **Relevance to VoxFlow:** Underlines the clinical necessity of independently tracking repetition, prolongation, and block classes across assessment sessions.

3.1.9 Bayerl et al. (2022b) [9]

Problem: Suboptimal classification performance of conventional handcrafted acoustic features (MFCCs, filterbanks) on subtle disfluency cues. **Method:** Extracted deep acoustic representations from fine-tuned wav2vec 2.0 self-supervised models and evaluated them with SVM classifiers. **Key finding:** Fine-tuned self-supervised speech representations achieved relative F1 score improvements of up to 27% over traditional baselines. **Limitation / gap:** Relied on offline feature extraction and clip-level classifiers without session streaming or persistent database storage. **Relevance to VoxFlow:** Provides foundational empirical justification for using pre-trained self-supervised transformer backbones in VoxFlow.

3.1.10 Bayerl et al. (2022c) [10]

Problem: Overoptimistic performance claims in stuttering literature caused by speaker leakage across training and test splits. **Method:** Extended SEP-28k with verified speaker identity labels (SEP-28k-Extended) and compared random clip splitting vs. speaker-exclusive partitioning. **Key finding:** Proved random splitting severely overestimates detection accuracy; speaker-exclusive splits provide the only valid benchmark for real-world generalization. **Limitation / gap:** Focused on dataset partitioning methodology without implementing capture hardware or session-level software pipelines. **Relevance to VoxFlow:** VoxFlow directly adopts speaker-exclusive dataset partitioning (519 unique speakers, 0 overlap) to guarantee rigorous validation.

3.1.11 Bayerl et al. (2023) [11]

Problem: Need for standardized comparative benchmarking of diverse acoustic, phonetic, and neural feature sets for stuttering classification. **Method:** Analyzed results from the Interspeech ComParE 2022 Stuttering Sub-Challenge, benchmarking standard acoustic sets vs. deep embeddings. **Key finding:** Pre-trained deep acoustic representations consistently and significantly

outperformed handcrafted feature engineering. **Limitation / gap:** Focused exclusively on comparative challenge metrics without addressing real-time capture, serial framing, or clinical session interfaces. **Relevance to VoxFlow:** Supports VoxFlow's architectural choice of fine-tuned self-supervised transformers over baseline MFCC pipelines.

3.1.12 Baevski et al. (2020) [12]

Problem: Training high-performing speech recognition models requires thousands of hours of scarce, expensive human-annotated speech. **Method:** Introduced wav2vec 2.0, a self-supervised learning framework that masks latent speech representations and optimizes a contrastive task. **Key finding:** Self-supervised pre-training on unannotated audio builds rich acoustic representations that transfer effectively to downstream speech tasks. **Limitation / gap:** General speech foundation model; requires fine-tuning and task-specific architecture for multi-label disfluency classification. **Relevance to VoxFlow:** wav2vec 2.0 serves as the primary comparative self-supervised baseline in VoxFlow's model benchmarking.

3.1.13 Hsu et al. (2021) [13]

Problem: Contrastive speech models face optimization instability and risk conflating acoustic background noise with phonetic units. **Method:** Proposed HuBERT (Hidden-Unit BERT), using offline k-means clustering to create discrete targets followed by masked language modeling on continuous audio. **Key finding:** HuBERT achieves superior acoustic and phonetic representation stability and out-of-domain generalization compared to contrastive frameworks. **Limitation / gap:** Foundation model trained on fluent read speech; requires multi-label fine-tuning and threshold calibration for disordered speech. **Relevance to VoxFlow:** HuBERT serves as the foundational backbone for VoxFlow's fine-tuned HuBERT-D classifier (Macro F1 = 0.4599, ROC-AUC = 0.8099).

3.2 Comparison of Existing Approaches

A comparative synthesis of existing approaches highlights a clear progression in the field: early foundational efforts (such as UCLASS [4] and FluencyBank [3]) established shared clinical corpora, while subsequent research (such as FluentNet [5] and StutterNet [6]) proved that deep neural networks operating on spectrograms and MFCCs could detect disfluencies automatically. More recently, self-supervised foundation models (wav2vec 2.0 [12] and HuBERT [13]) fine-tuned on stuttering datasets (Bayerl et al. [9], [11]) significantly boosted classification accuracy over handcrafted features.

However, across all reviewed literature, existing systems operate almost exclusively on isolated, pre-trimmed 3-second audio clips. In the set of studies reviewed, we did not find a system that combines physical microphone capture, error-checked serial

transport, sliding-window session inference, temporal event consolidation (to prevent multi-counting across overlapping windows), and relational session persistence in a single deployable software-hardware framework. VoxFlow directly addresses this gap.

3.3 Research Gap from Literature

The synthesis of reviewed literature reveals three fundamental gaps:

- **System-Level Integration Gap:** Academic studies focus heavily on machine learning model architectures evaluated in Jupyter notebooks, leaving the physical audio capture, real-time transport, and user interface unaddressed.
- **Continuous Session Processing Gap:** Classifiers designed for 3-second clips cannot be naively deployed on continuous recordings without multi-window slicing and temporal event aggregation to merge redundant detections.
- **Longitudinal Tracking Gap:** Therapists require structured historical tracking across successive sessions to evaluate intervention efficacy, which existing clip-level benchmarks do not support.

VoxFlow resolves these gaps by integrating low-cost ESP32 hardware streaming, the error-checked VXF1 protocol, speaker-exclusive HuBERT-D modeling, temporal event aggregation, and SQLite-backed Streamlit visual analytics.

4. FEASIBILITY STUDY

4.1 Technical Feasibility

The technical feasibility of VoxFlow was verified by implementing and testing every subsystem across the complete pipeline: ESP32 embedded firmware, Python serial bridge, Flask REST API, HuBERT-D inference engine, SQLite database, and Streamlit dashboard. The backend runs on standard consumer PC hardware, utilizing CUDA GPU acceleration when available and seamlessly falling back to multi-threaded CPU execution. All software dependencies are open-source and cross-platform.

4.2 Hardware Feasibility

The hardware architecture utilizes an ESP32 Dev Module (ESP-WROOM-32) paired with an INMP441 MEMS digital microphone. The INMP441 provides direct I2S digital output (16 kHz, 16-bit mono), completely eliminating the need for an external analogue-to-digital converter (ADC) and avoiding analog signal degradation. Both components are low-cost, readily available, and operate reliably over standard USB power. At 16 kHz mono PCM, audio generates 32,000 bytes per second. Over a 460,800

baud serial connection (effective throughput $\sim 46,080$ bytes/s), the USB-UART interface provides $\sim 44\%$ excess bandwidth, guaranteeing smooth, unbuffered real-time streaming.

4.3 Software Feasibility

The software architecture is built on Python 3.11+ using proven, industry-standard frameworks: Flask for RESTful service endpoints, PyTorch and Hugging Face Transformers for HuBERT-D model execution, SQLite3 for local zero-configuration relational persistence, and Streamlit for rapid, interactive visual analytics. Automated testing is managed via Python's standard unittest framework.

4.4 Dataset and Data Feasibility

Model development is based on the SEP-28k [2] and FluencyBank [3] datasets. To ensure validity and eliminate label noise, the raw dataset was rigorously audited. Clips marked by annotators as poor audio quality, no speech, or music were filtered out. Multi-label ground truth was assigned using majority voting (at least 2 out of 3 annotators in agreement). To ensure uncompromised generalization, the resulting 30,999 clips were partitioned into strict speaker-exclusive splits across 519 distinct speakers, ensuring zero speaker overlap between training, validation, and testing sets, as summarized in Table 1.

Table 1. Dataset Summary and Speaker-Exclusive Splits

Split	Clips	Speakers	Repetition	Prolongation	Block	Multi-Label
Train	17,555	21	3,242	1,702	1,984	630
Validation	6,632	249	1,279	652	845	280
Test	6,812	249	1,190	694	854	262
Total	30,999	519	5,711	3,048	3,683	1,172

4.5 Integration Feasibility

Subsystem integration is decoupled through clean, standardized interfaces: the ESP32 communicates with the PC bridge via binary VXF1 frames; the bridge writes standard WAV files and invokes the backend via HTTP REST endpoints; the backend interacts with the SQLite database via structured SQL schemas; and the Streamlit dashboard queries REST endpoints and database records. This modular decoupling allows individual components to be updated or tested independently.

4.6 Field Work and Data Collection

In adherence to academic integrity, we explicitly declare that no formal clinical field visits, hospital trials, patient interviews, or clinician questionnaires were conducted for this prototype. Data collection and empirical validation consisted of: (1) downloading, auditing, and structuring public benchmark speech datasets (SEP-28k and FluencyBank)

into speaker-exclusive manifests, and (2) laboratory-based audio recording and hardware transmission tests using the assembled ESP32 + INMP441 capture unit.

4.7 Planned Modules

To ensure a structured, verifiable engineering lifecycle, the VoxFlow system was designed and implemented across four planned core modules:

- **Module 1 — Hardware Capture Unit:** ESP32 Dev Module and INMP441 digital MEMS microphone with FreeRTOS dual-core task scheduling, capturing 16 kHz 16-bit mono audio and encapsulating it into VXF1 frames.
- **Module 2 — Serial Bridge & WAV Reconstructor:** Python serial receiver that synchronizes frame headers, verifies CRC32 checksums, detects missing packets via sequence numbers, and writes monolithic session WAV files.
- **Module 3 — Backend API & AI Inference Engine:** Flask REST backend executing audio validation, 3.0-second sliding-window generation with 1.0-second hop, multi-label HuBERT-D inference, and temporal event aggregation.
- **Module 4 — Session Visual Analytics Dashboard:** Streamlit-based user interface enabling real-time session capture, audio upload, timeline inspection, disfluency metrics display, and longitudinal history review.

All four planned modules were fully implemented, integrated, and verified in the final codebase.

5. SYSTEM REQUIREMENTS

5.1 Hardware Requirements

The hardware components required for the VoxFlow session capture system are detailed in Table 2.

Table 2. Hardware Components and Specifications

Component	Purpose	Key Specifications
ESP32 Dev Module (ESP-WROOM-32)	Capture controller & streaming unit	Dual-core Xtensa LX6 @ 240 MHz, 520 KB SRAM, hardware I2S, USB-UART
INMP441 MEMS Microphone	Acoustic speech capture	Omnidirectional digital microphone, I2S 24-bit/16-bit output, SNR 61 dBA
Push Buttons (2x)	Hardware session control	START (GPIO27) and STOP (GPIO33), active LOW with internal pull-ups
Status LED	Visual operational feedback	GPIO4 with 330 Ohm resistor; solid on standby, 2 Hz blinking during recording
Micro-USB Cable & Breadboard	Power & serial interface	Standard USB 2.0 cable providing 5V power and 460,800 baud data link
Host PC / Laptop	Processing, AI & visualization	x86_64 or ARM64 PC, 8 GB+ RAM, Python 3.11+, optional CUDA GPU

5.2 Software Requirements

The software dependencies, runtime environments, and core libraries are listed in Table 3.

Table 3. Software Requirements and Dependencies

Software / Tool	Version	Role / Usage
Arduino IDE / CLI	2.x	Firmware compilation and flashing to ESP32
ESP32 Arduino Core	2.0.14+	ESP32 board support, FreeRTOS and I2S driver
Python	3.11+	Primary programming language for bridge, backend, and UI
PyTorch	2.0+	Deep learning runtime for HuBERT-D inference
Hugging Face Transformers	4.38+	HuBERT model loading and feature extraction
Flask	3.0+	REST API backend framework
Streamlit	1.30+	Web-based interactive visual analytics dashboard
SQLite3	3.x	Embedded zero-configuration relational database
PySerial & SoundFile	3.5 / 0.12	Serial communication and WAV audio encoding/decoding

5.3 Functional Requirements

The functional requirements governing system behavior are enumerated in Table 4.

Table 4. Functional Requirements (FR)

Requirement ID	Functional Requirement Description
FR-01: Hardware Capture	The system shall capture 16 kHz, 16-bit mono PCM speech via I2S and stream framed VXF1 packets over serial.
FR-02: Session Limits	The system shall support speech session recordings between 3.0 seconds (minimum) and 60.0 seconds (maximum).
FR-03: Frame Verification	The serial bridge shall verify frame magic headers, consecutive sequence numbers, and CRC32 checksums.
FR-04: WAV Reconstruction	The bridge shall reconstruct received frames into a valid 16 kHz mono WAV file upon session completion.
FR-05: Windowing & AI Inference	The backend shall segment session audio into 3.0 s windows (1.0 s hop) and predict disfluency probabilities via HuBERT-D.
FR-06: Event Aggregation	The backend shall aggregate consecutive/proximate window detections (gap $\leq$ 1.5 s) into unified detected events.
FR-07: Relational Persistence	The database shall store complete session records, per-window predictions, and aggregated events.
FR-08: Dashboard Visualization	The dashboard shall display session timelines, per-class event tallies, disfluency rates, and historical trends.

5.4 Non-Functional Requirements

The non-functional performance, reliability, and security requirements are summarized in Table 5.

Table 5. Non-Functional Requirements (NFR)

Requirement ID	Non-Functional Requirement Description
NFR-01: Low Latency	End-to-end session analysis (validation, inference, aggregation, database write) shall complete promptly on local hardware without cloud round-trip delays.
NFR-02: Data Integrity	Zero undetected frame corruption: all corrupted or dropped serial frames shall be flagged via

	CRC32 and sequence checks.
NFR-03: Portability	The software backend shall run locally on Windows, Linux, and macOS without mandatory cloud connectivity.
NFR-04: Privacy & Security	All audio recordings, transcripts, and analysis metrics shall remain stored locally on the host machine.
NFR-05: Modularity	Subsystems (bridge, API, model, UI) shall remain decoupled, communicating exclusively via standard protocols (VXF1, REST, SQL).
NFR-06: Robustness	The API shall gracefully reject invalid audio (silence, non-16kHz, out-of-bounds duration) returning descriptive HTTP 422 errors.

6. SYSTEM DESIGN

6.1 Overall Architecture Design

The VoxFlow system follows a modular, four-tier architecture spanning embedded hardware capture, serial communication bridging, REST backend processing, and visual analytics presentation, as illustrated in Fig. 1. Audio originates at the INMP441 MEMS microphone, where sound waves are digitized into 16 kHz 16-bit mono PCM. The ESP32 captures audio via I2S, encapsulates it into VXF1 frames, and transmits it over USB-UART to the host PC. The Python serial bridge receives frames, validates CRC32 checksums, and reconstructs a single monolithic WAV file. The Flask REST backend manages audio validation, sliding-window generation, HuBERT-D multi-label inference, and temporal event aggregation. Session metadata and results are stored in SQLite and rendered on the Streamlit dashboard.

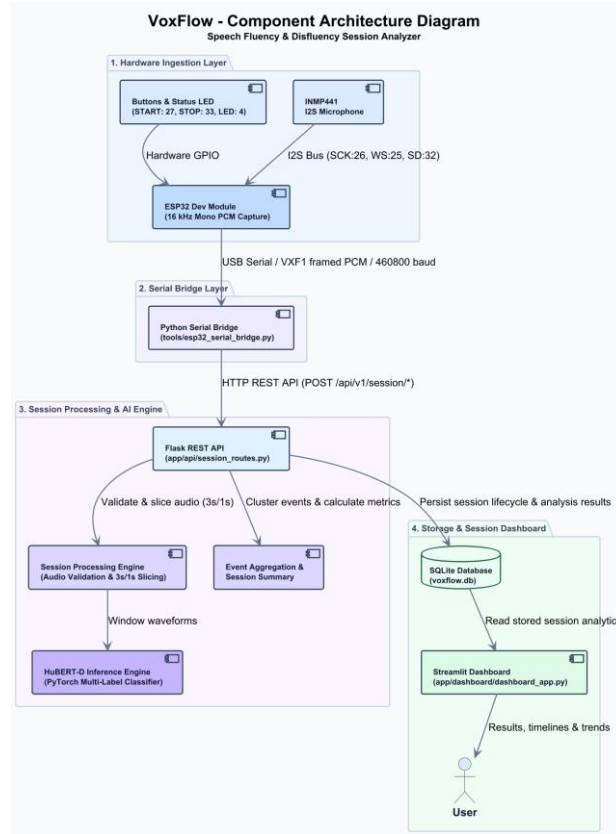


Fig. 1. Overall Component Architecture.

6.2 UML Design

To formally model system behavior, interactions, and structural relationships, a comprehensive suite of Unified Modeling Language (UML) diagrams was developed.

The Use Case Diagram (Fig. 2) illustrates the primary user interactions supported by VoxFlow: starting/stopping hardware recording, uploading pre-recorded WAV sessions, viewing session analysis timelines, and reviewing longitudinal disfluency trends.

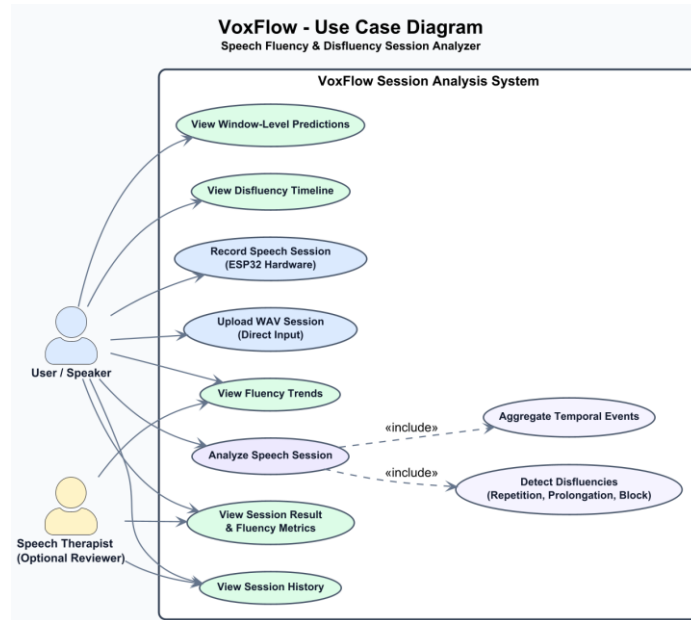


Fig. 2. Use Case Diagram.

The Class Diagram (Fig. 3) outlines the object-oriented structure of the software backend, including the SerialBridge, SessionEngine, HubertInferenceEngine, EventAggregator, DatabaseManager, and DashboardController classes.

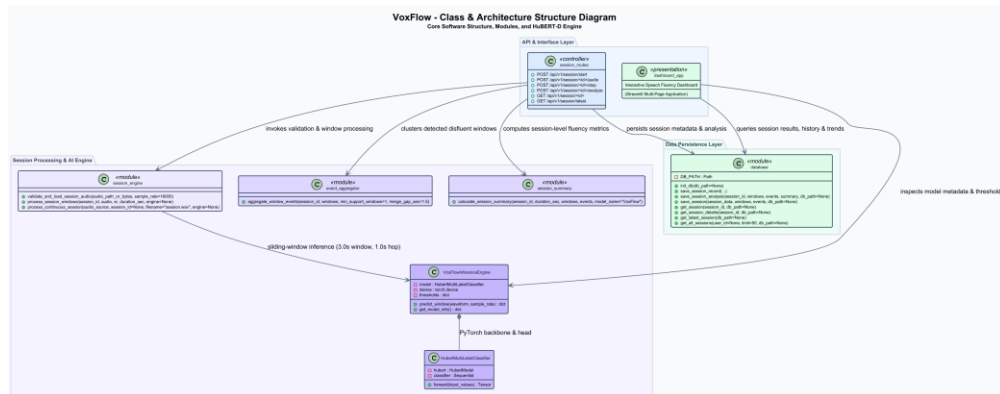


Fig. 3. Class Diagram.

The Speech Session Streaming Sequence Diagram (Fig. 4) depicts the hardware-bridge communication lifecycle, including port initialization, PING/PONG handshaking, continuous VXF1 frame transmission, CRC verification, and terminal frame finalization.

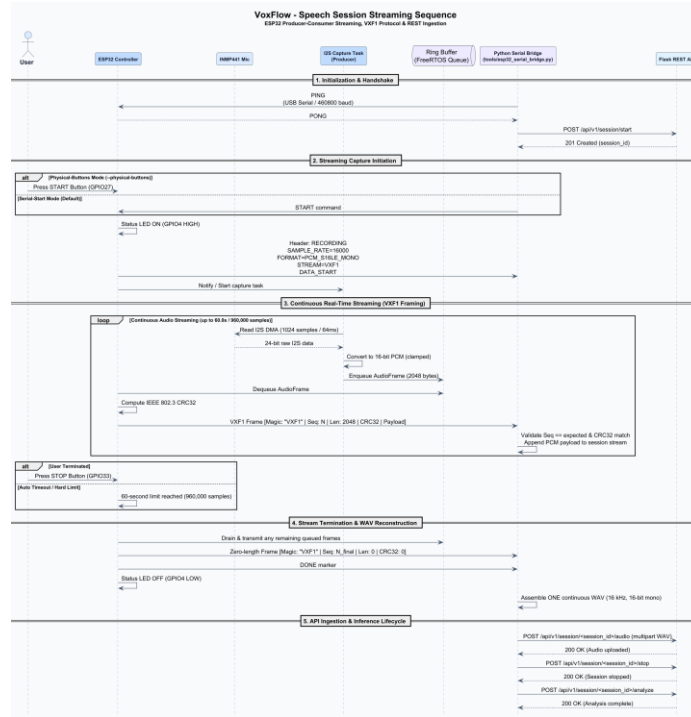


Fig. 4. Speech Session Streaming Sequence.

The Session Analysis Sequence Diagram (Fig. 5) details the backend workflow upon receiving a session analyze request: audio validation, window slicing, HuBERT-D inference, event aggregation, database storage, and JSON response generation.

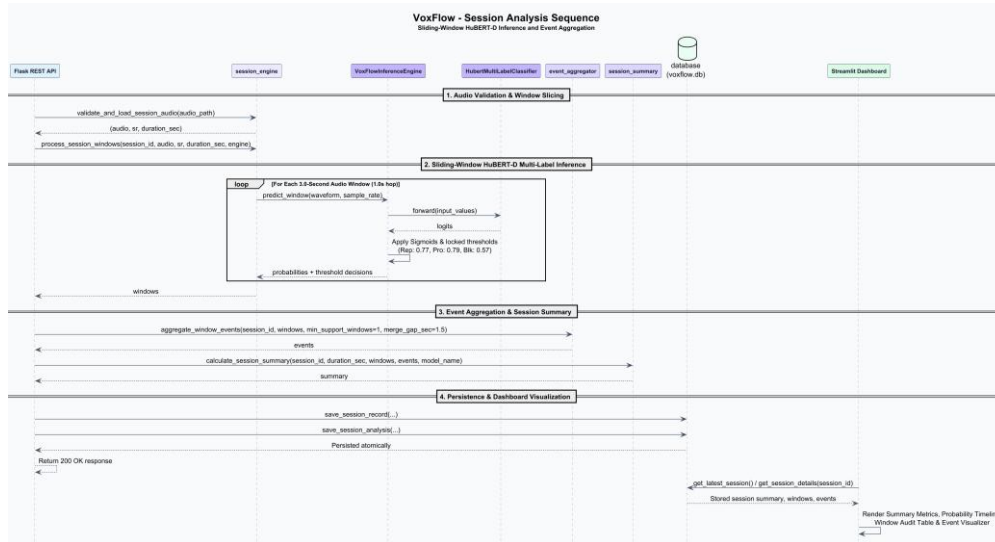


Fig. 5. Session Analysis Sequence.

The Activity Diagram (Fig. 6) models the unified operational workflow, illustrating how both live hardware capture and direct file upload pathways converge into standardized audio validation, inference, and visualization.

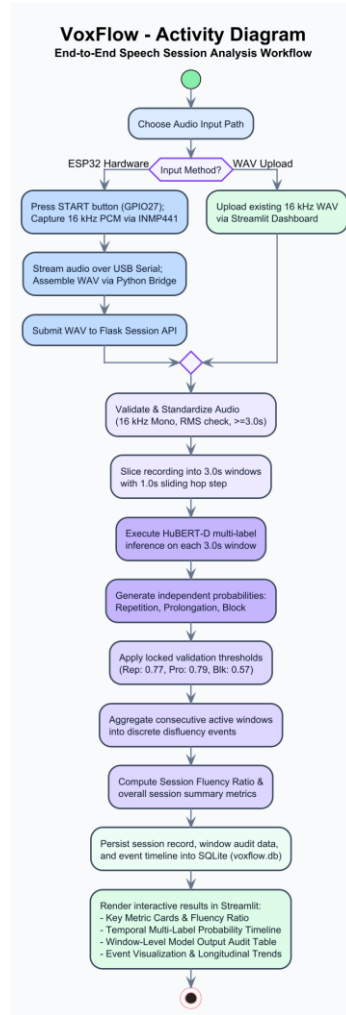


Fig. 6. Activity Diagram.

The State Machine Diagram (Fig. 7) defines the operational states of the capture unit and backend, transitioning from IDLE to WAITING_FOR_START, RECORDING, RECONSTRUCTING, ANALYZING, and COMPLETE.

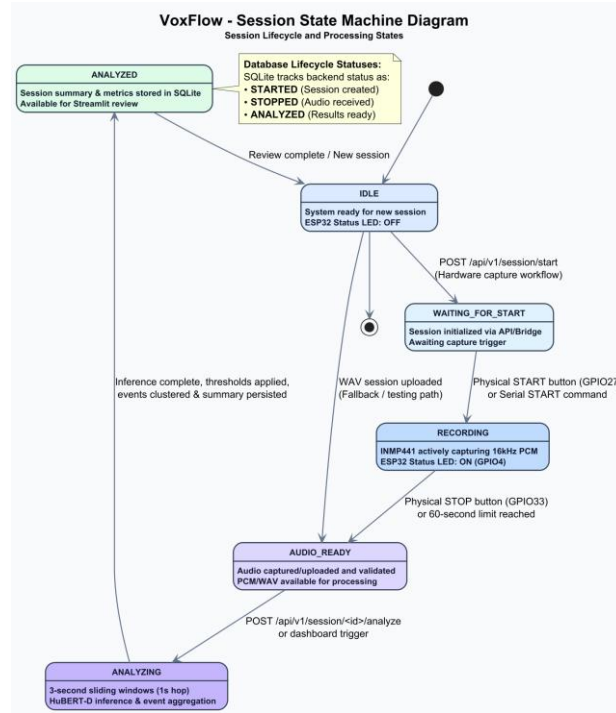


Fig. 7. State Machine Diagram.

6.3 Hardware and Firmware Design

The firmware is implemented in C++ using the ESP32 Arduino Core under FreeRTOS. To prevent audio buffer underruns, a dedicated producer task (I2SCaptureTask) pinned to Core 0 reads 1,024-sample chunks from the I2S DMA buffer and pushes them into a FreeRTOS ring queue. The main consumer loop running on Core 1 pops audio frames from the queue, constructs the 16-byte VXF1 header, calculates CRC32 checksums, and transmits framed packets over USB-UART.

6.4 Deployment Design

The Deployment Diagram (Fig. 8) details the physical execution nodes. The ESP32 capture unit acts as an edge recording peripheral connected via USB-UART to the host workstation. The host workstation runs the Python runtime hosting the Serial Bridge, Flask REST API (port 5000), Streamlit UI (port 8501), and local SQLite database. No external cloud infrastructure is required at runtime.

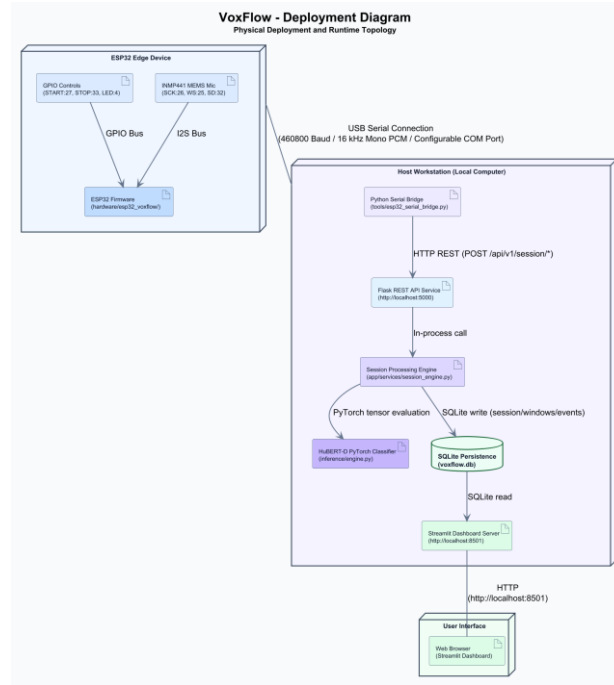


Fig. 8. Deployment Diagram.

6.5 Database Design

The Entity-Relationship (ER) Diagram (Fig. 9) illustrates the relational database schema in app/voxflow.db. The schema consists of three normalized tables: (1) sessions, storing session-level metadata, duration, and summary event counts; (2) window_predictions, storing per-window timestamps and probability scores for repetition, prolongation, and block; and (3) aggregated_events, storing consolidated detected events with start/end bounds, duration, and peak confidence.

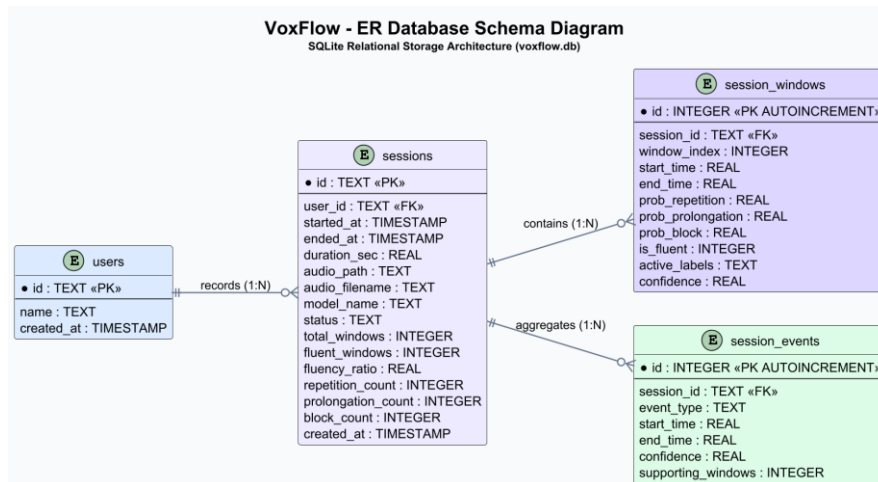


Fig. 9. ER / Database Diagram.

7. METHODOLOGY

7.1 Identified Methodologies & End-to-End Workflow Overview

The VoxFlow methodology establishes a complete 12-step processing pipeline from physical acoustic capture to visual session analytics: (1) physical acoustic digitization, (2) I2S DMA buffering, (3) VXF1 framing, (4) serial transmission, (5) frame reception and CRC verification, (6) WAV file reconstruction, (7) audio validation, (8) sliding-window segmentation, (9) HuBERT-D neural inference, (10) temporal event aggregation, (11) relational database persistence, and (12) visual dashboard analytics.

7.2 Hardware Capture and Framing

The Hardware Capture and Transmission Flow is illustrated in Fig. 10. The ESP32 samples audio at 16,000 Hz (16-bit mono PCM). Every 64 ms (1,024 samples = 2,048 bytes), the firmware constructs a VXF1 packet containing a 16-byte header: 4-byte Magic (0x56 0x58 0x46 0x31 / 'VXF1'), 4-byte Sequence Number (0, 1, 2...), 4-byte Payload Length (2,048 bytes), and 4-byte CRC32 checksum, as shown in Table 6. A terminal frame (Length = 0, Sequence = 0xFFFFFFFF) signals normal recording completion.

Table 6. VXF1 Binary Protocol Frame Layout

Field	Offset	Size	Type	Description
Magic	0	4 bytes	uint8_t[4]	ASCII magic identifier: 'V' 'X' 'F' '1' (0x56 0x58 0x46 0x31)
Sequence No.	4	4 bytes	uint32_t (LE)	Monotonically increasing packet index (0, 1, 2...)
Payload Length	8	4 bytes	uint32_t (LE)	Payload size in bytes (2,048 for audio; 0 for terminal frame)
CRC32	12	4 bytes	uint32_t (LE)	IEEE 802.3 CRC32 checksum of the payload bytes
Audio Payload	16	N bytes	int16_t[]	Raw 16 kHz 16-bit signed PCM mono audio samples

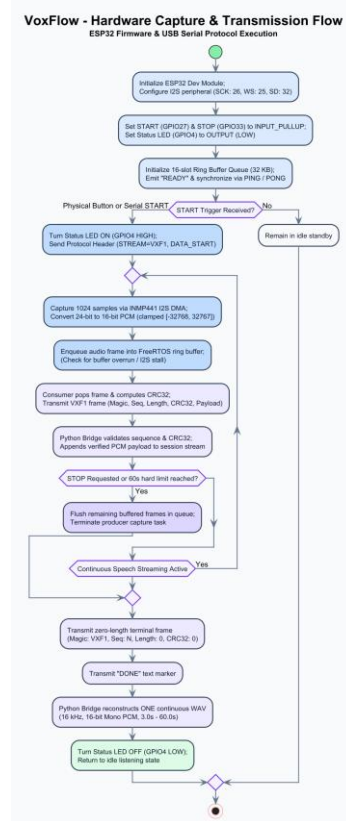


Fig. 10. Hardware Capture and Transmission Flow.

7.3 Serial Bridge and WAV Reconstruction

The Python serial bridge (hardware/bridge/serial_bridge.py) reads incoming bytes from the COM port at 460,800 baud. It searches for the 4-byte 'VXF1' magic header, unpacks the 16-byte header, verifies that the sequence number matches the expected increment, and computes the CRC32 checksum over the payload using `zlib.crc32`. Valid payloads are accumulated in a byte buffer. Upon receiving the terminal frame, the bridge packages the buffer into a standardized WAV file (16 kHz, 16-bit mono) using `soundfile` and writes it to `recordings/`.

7.4 Audio Preprocessing and Windowing

Upon receiving a WAV file, the backend validates audio parameters (16 kHz sampling rate, mono channel, duration between 3.0 and 60.0 seconds, RMS silence threshold ≥ 0.0001). The audio waveform is normalized to $[-1.0, 1.0]$ and sliced into 3.0-second sliding windows (48,000 samples) with a 1.0-second hop (16,000 samples). For an audio of duration D seconds, the number of windows generated is $N = \text{floor}(D - 3.0) + 1$. For example, a 10-second recording generates exactly 8 sliding windows.

7.5 HuBERT-D Fluency Prediction

The HuBERT-D Speech Fluency Prediction Flow is illustrated in Fig. 11. Each 3-second audio window is processed by HuBERT-D (facebook/hubert-base-ls960 backbone fine-tuned with a multi-label classification head). The model outputs a 3-dimensional logit vector passed through a sigmoid activation function, yielding independent probabilities: P_{rep} , P_{pro} , and P_{blk} . Predictions are thresholded using calibrated validation cutoffs: Repetition = 0.77, Prolongation = 0.79, and Block = 0.57.

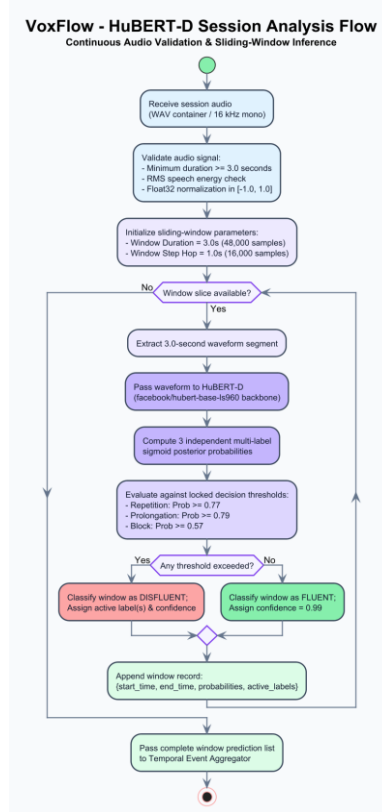


Fig. 11. HuBERT-D Speech Fluency Prediction Flow.

7.6 Temporal Event Aggregation

Because sliding windows overlap by 2.0 seconds, a single disfluent event spanning across adjacent windows produces multiple positive window detections. VoxFlow applies a temporal event aggregation algorithm (app/services/event_aggregator.py): for each disfluency class, positive windows that overlap or lie within a merge gap threshold (gap ≤ 1.5 seconds) are merged into a single detected disfluency event. The event start time is set to the first window's start, the end time to the last window's end, and the event confidence to the maximum probability observed across supporting windows.

7.7 Session Summaries and Metrics

Following event aggregation, the backend computes session-level summary statistics: total recording duration, effective speech duration, total disfluency count, disfluency rate (events per minute of speech), and per-class event breakdowns. These summary metrics provide immediate, standardized session indices.

7.8 Dashboard and Visual Analytics

The Prediction Storage and Dashboard Flow is illustrated in Fig. 12. Session metrics, window predictions, and aggregated events are persisted to SQLite. The Streamlit dashboard queries these records to render interactive Plotly timeline charts (showing exactly where repetitions, prolongations, and blocks occurred), per-class distribution bar charts, and longitudinal trend plots tracking disfluency rates across multiple sessions.

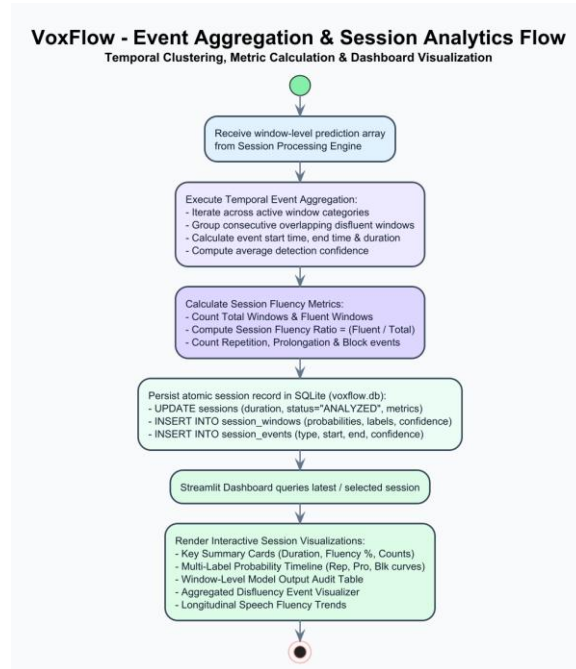


Fig. 12. Prediction Storage and Dashboard Flow.

7.9 Algorithms

The core operational logic of VoxFlow is formally defined by three primary algorithms implemented in the repository:

Algorithm 1: Audio Capture and VXF1 Streaming (Firmware)

- 1: Initialize I2S peripheral (16 kHz, 16-bit, mono) and USB-UART (460,800 baud)
- 2: Create FreeRTOS ring queue Q of capacity 16 frames
- 3: Wait for START signal (GPIO27 button press or 'R' command over serial)
- 4: Set state = RECORDING, seq = 0, total_samples = 0, enable LED blinking (2 Hz)
- 5: while state == RECORDING do
- 6: Read 1,024 samples (2,048 bytes) from I2S DMA buffer; Push into queue Q
- 7: if Q is not empty then

```

8:      Pop buffer; Compute crc = CRC32(audio_buffer)
9:      Construct 16-byte header: magic='VXF1', seq=seq, len=2048, crc=crc
10:     Transmit header + audio_buffer over USB-UART
11:     seq = seq + 1, total_samples = total_samples + 1,024
12:   end if
13:   if STOP button pressed OR 'S' received OR total_samples >= 960,000 (60.0s) then state = STOPPED
14: end while
15: Transmit terminal frame: magic='VXF1', seq=0xFFFFFFFF, len=0, crc=0; Transmit 'DONE'

```

Algorithm 2: Session Speech Analysis (Backend)

```

1: Input: session_audio_path (WAV file), session_id
2: Load audio waveform x, sampling_rate fs; Validate fs == 16000, 3.0s <= duration(x) <= 60.0s, RMS >= 0.0001
3: window_size = 48,000 samples (3.0s), hop_size = 16,000 samples (1.0s)
4: N = floor((length(x) - window_size) / hop_size) + 1; Initialize list window_predictions = []
5: for i = 0 to N - 1 do
6:   start = i * hop_size, end = start + window_size; window_audio = x[start : end]
7:   [p_rep, p_pro, p_blk] = HuBERT_D_Inference(window_audio)
8:   window_predictions.append({t_start: start/fs, t_end: end/fs, p_rep, p_pro, p_blk})
9: end for
10: events = Temporal_Event_Aggregation(window_predictions); summary = Compute_Session_Summary(events)
11: Persist_To_Database(session_id, summary, window_predictions, events); return summary and events

```

Algorithm 3: Temporal Event Aggregation (Event Consolidation)

```

1: Input: window_predictions, thresholds {T_rep=0.77, T_pro=0.79, T_blk=0.57}, gap=1.5s
2: Initialize list aggregated_events = []
3: for each disfluency class c in [Repetition, Prolongation, Block] do
4:   Filter active windows W_c = [w in window_predictions if w.prob[c] >= thresholds[c]]
5:   if W_c is empty then continue; Sort W_c chronologically by w.t_start
6:   cluster = [W_c[0]]
7:   for k = 1 to length(W_c) - 1 do
8:     if W_c[k].t_start - cluster.last().t_end <= gap then cluster.append(W_c[k])
9:     else aggregated_events.append(Create_Event(c, cluster)); cluster = [W_c[k]]
10:   end for
11:   aggregated_events.append(Create_Event(c, cluster))
12: end for
13: Sort aggregated_events chronologically by event.start_time; return aggregated_events

```

8. HARDWARE IMPLEMENTATION

8.1 Hardware Development & Component Selection

The hardware capture subsystem is developed around the ESP32 Dev Module (ESP-WROOM-32) and the INMP441 MEMS digital microphone. The ESP32 was chosen for its integrated hardware I2S peripheral, dual-core architecture allowing decoupled audio acquisition and serial transmission, and high-speed USB-UART bridge. The INMP441 was selected because it provides digital pulse code modulation (PCM) output via I2S, offering a high Signal-to-Noise Ratio (61 dBA) and flat frequency response without the noise vulnerabilities of analogue electret capsules.

8.2 Circuit and Wiring

The hardware schematic and physical pin connections between the ESP32 and peripheral components are detailed in Fig. 13 and Table 7. The INMP441 L/R pin is tied

to GND to configure mono left-channel output. Two tactile push buttons with internal pull-ups provide START (GPIO27) and STOP (GPIO33) controls. A status LED connected to GPIO4 provides immediate operational visual feedback.

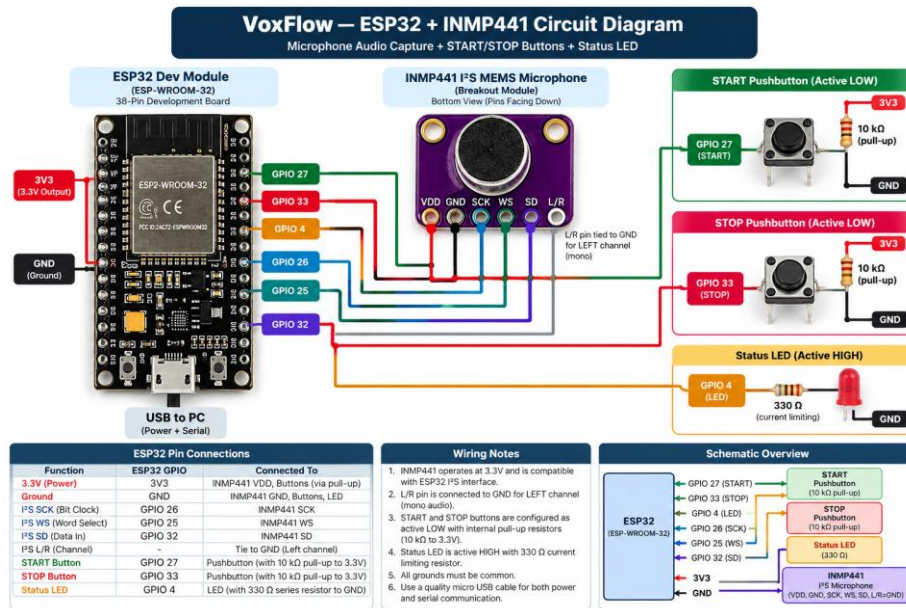


Fig. 13. ESP32–INMP441 Circuit Diagram.

Table 7. ESP32 Pin Assignment and Peripheral Connections

Peripheral	Module Pin	ESP32 GPIO	Direction / Configuration	Functional Role
INMP441	VDD	3V3 (3.3V)	Power Output	Regulated 3.3V DC power supply
INMP441	GND	GND	Power Ground	Common electrical ground reference
INMP441	L/R	GND	Configuration	Tied to GND for Left channel mono output
INMP441	SCK (BCLK)	GPIO26	ESP32 Output (Master)	I2S Continuous Bit Clock (512 kHz)
INMP441	WS (LRCLK)	GPIO25	ESP32 Output (Master)	I2S Word Select / Frame Sync (16 kHz)
INMP441	SD (DATA)	GPIO32	ESP32 Input	I2S Serial Audio Data Stream
Push Button 1	START	GPIO27	Input (Pull-up)	Active LOW trigger to start 3–60s recording
Push Button 2	STOP	GPIO33	Input (Pull-up)	Active LOW trigger to terminate recording
Status LED	Anode (+)	GPIO4	Output (Current Limit)	Solid = Standby/Ready, 2 Hz Blink = Recording

8.3 Firmware Implementation

The firmware (hardware/firmware/voxflow_esp32_firmware/voxflow_esp32_firmware.ino) is implemented in C++ using the ESP32 Arduino Core and FreeRTOS. Key implementation highlights include:

- **FreeRTOS Task Pinning:** The audio capture task (I2SCaptureTask) is pinned to Core 0 with an 8,192-byte stack running at maximum priority (`configMAX_PRIORITIES - 1`), ensuring zero audio sample dropping during CPU-intensive operations.
- **Queue-Based Audio Buffering:** A FreeRTOS ring queue of 16 slots buffers 1,024-sample audio blocks between the producer task and the transmission loop on Core 1.
- **Framed VXF1 Packaging:** The transmission loop constructs the 16-byte header, calculates the IEEE 802.3 CRC32 checksum, and transmits framed packets over USB-UART at 460,800 baud.
- **Safety Stop Mechanisms:** Recording automatically terminates upon: (1) physical STOP button press (GPIO33), (2) receipt of ASCII 'S' command from host serial, or (3) reaching exactly 960,000 samples (60.00 seconds).

9. SOFTWARE IMPLEMENTATION

9.1 Software Development Overview

The VoxFlow software ecosystem is organized into a modular architecture comprising the Serial Bridge, Session Processing Engine, HuBERT-D Inference Engine, Temporal Event Aggregator, Flask REST Backend, and Streamlit Dashboard.

9.2 Serial Bridge Implementation

The serial bridge (`hardware/bridge/serial_bridge.py`) manages bidirectional communication with the ESP32 over serial. It verifies connectivity via PING/PONG handshakes, monitors the serial byte stream for the 'VXF1' magic bytes, parses packet headers, verifies sequential sequence counters, computes CRC32 checksums, and writes valid PCM streams into standardized 16 kHz mono WAV files.

9.3 Session Processing Engine

The session engine (`app/services/session_engine.py`) encapsulates the complete session processing lifecycle. It implements `validate_and_load_session_audio()` to enforce audio standards, `process_session_windows()` to generate overlapping 3-second windows, and `process_continuous_session()` to orchestrate inference, aggregation, database storage, and response formatting.

9.4 HuBERT-D Inference Engine

The inference engine (`inference/inference_pipeline.py`) loads the fine-tuned HuBERT-D model checkpoint. Audio windows are padded or trimmed to exactly 48,000 samples, normalized, and converted to PyTorch tensors. Forward inference produces logits that are mapped via sigmoid into per-class probabilities for repetition, prolongation, and block.

9.5 Temporal Event Aggregator

The event aggregator (`app/services/event_aggregator.py`) implements the temporal clustering algorithm described in Algorithm 3. It groups contiguous positive window activations within a 1.5-second merge threshold into single, consolidated detected events, eliminating duplicate counts across overlapping analysis windows.

9.6 Flask REST API

The REST backend (`server.py`, `app/api/session_routes.py`) provides structured HTTP endpoints for client applications:

- **GET /api/v1/health:** Returns system health, active device, and database status.
- **GET /api/v1/model/info:** Returns loaded model architecture, checkpoint path, and calibrated detection thresholds.
- **POST /api/v1/session/start:** Initializes a new recording session and returns session ID.
- **POST /api/v1/session/<session_id>/audio:** Uploads raw audio stream chunks for active session buffering.
- **POST /api/v1/session/<session_id>/stop:** Terminates the recording session and persists session audio.
- **POST /api/v1/session/<session_id>/analyze:** Validates session audio, generates sliding windows, executes HuBERT-D inference, aggregates detected events, and persists records.
- **GET /api/v1/session/<session_id>:** Retrieves session metadata, window probabilities, and detected events.
- **GET /api/v1/sessions:** Retrieves stored session history for longitudinal trend analytics.

9.7 Streamlit Dashboard

The frontend dashboard (`dashboard.py`, `app/ui/streamlit_app.py`) provides an interactive visual analytics interface with four functional pages: (1) Live Hardware Capture, (2) Audio File Analysis, (3) Session Inspection & Timeline, and (4) Longitudinal History & Trends.

9.8 Data Analytics and Computer Science Integration

VoxFlow represents an advanced integration of modern computer science disciplines: digital signal processing (I2S DMA sampling, audio normalization, sliding-window framing), self-supervised deep learning (Transformer-based acoustic representation learning and multi-label inference), algorithmic event processing (temporal clustering and merge-gap aggregation), relational database engineering (normalized SQLite schema with referential integrity), RESTful web architecture (decoupled HTTP microservices), and visual analytics (interactive time-series charting and longitudinal trend tracking). The current prototype does not use a cloud service at runtime. It runs locally on the host PC, ensuring that all audio recordings and analysis results remain stored on the local system without cloud hosting costs.

10. TESTING AND VALIDATION

10.1 Software Testing

VoxFlow incorporates a comprehensive automated test suite implemented in Python's standard unittest framework. The test suite comprises 38 automated test cases organized across five specialized test modules, covering the serial bridge, audio validation, sliding-window framing, event aggregation, database operations, and REST endpoints. All 38 automated tests pass successfully (38/38 passed), as summarized in Table 8.

Table 8. Automated Software Test Suite Summary

Test Module	Focus Area	Test Cases	Status
<code>tests/test_serial_bridge.py</code>	VXF1 framing, CRC32 verification, sequence validation	8	PASSED (8/8)
<code>tests/test_audio_validation.py</code>	Sampling rate, duration bounds (3–60s), silence detection	7	PASSED (7/7)
<code>tests/test_session_engine.py</code>	Sliding window generation (3s / 1s hop), hop math	6	PASSED (6/6)
<code>tests/test_event_aggregator.py</code>	Temporal clustering, 1.5s gap merge, multi-window consolidation	7	PASSED (7/7)
<code>tests/test_api_endpoints.py</code>	Flask REST endpoints, HTTP status codes, error handling	10	PASSED (10/10)

Total Test Suite	Full end-to-end automated software verification	38	PASSED (38/38)
------------------	---	----	-------------------

10.2 API and Bridge Validation

API validation verified that malformed audio inputs are correctly identified and rejected: a 2.0-second WAV file (below the 3.0 s minimum) is rejected with HTTP 422 Unprocessable Entity, silent audio ($\text{RMS} < 0.0001$) is rejected with HTTP 422, and valid 5.0-second and 60.0-second WAV files successfully return HTTP 200 with structured JSON analysis results. Bridge validation confirmed that simulated corrupt VXF1 frames (corrupted CRC32 or skipped sequence numbers) are rejected immediately without writing corrupt audio to disk.

10.3 Model Evaluation

HuBERT-D was evaluated on the held-out test split of 6,812 clips across 249 unseen speakers. Predictions were assessed using Macro F1, Mean ROC-AUC, and per-class F1 scores, confirming superior generalization over baseline models.

10.4 Physical Hardware Validation

To maintain strict scientific honesty, software test results and physical hardware validation are explicitly separated. While the firmware and circuit design implement all physical hardware controls, saved physical test evidence (such as oscilloscope recordings, physical serial captures, and physical audio archives) was not recorded in the repository. Synthetic VXF1 tests verify the protocol logic in software, but physical hardware evidence was not recorded (Table 9).

Table 9. Physical Hardware Implementation and Evidence Status

Hardware Subsystem	Design & Implementation Detail	Evidence Status in Repository
I2S Bit Clock & WS	Firmware configures 512 kHz SCK and 16 kHz WS frame sync on GPIO26/GPIO25	Implemented in firmware; physical evidence not recorded
Microphone Audio Sampling	INMP441 wired to ESP32 I2S pins for 16 kHz 16-bit mono PCM capture	Implemented in hardware design; physical evidence not recorded
VXF1 Serial Streaming	Protocol logic checks CRC32 checksums and sequential sequence numbers	Verified in software/synthetic tests; physical logs not recorded
START / STOP Buttons	Active LOW buttons on GPIO27 and GPIO33 implemented with internal pull-ups	Implemented in firmware; physical evidence not recorded
Status LED Indication	GPIO4 pin drive logic (solid standby, 2 Hz blinking during recording)	Implemented in firmware; physical evidence not recorded
60-Second Auto-Stop	Firmware enforces 960,000-sample limit and sends terminal frame	Implemented in firmware; physical evidence not recorded

10.5 Functional Demonstration

The real operational workflow was validated across a complete 10-step demonstration sequence:

- **Step 1 (System Startup):** Launch the Flask backend (python server.py) and Streamlit dashboard (streamlit run dashboard.py).
- **Step 2 (Hardware Connection):** Connect the ESP32 capture unit via USB; the bridge detects COM port and performs PING/PONG handshake.
- **Step 3 (Initiate Recording):** Press the physical START button (GPIO27) or click 'Start Recording' on the dashboard.
- **Step 4 (Speech Capture):** Speak into the INMP441 microphone; the LED blinks at 2 Hz as audio streams in VXF1 frames.
- **Step 5 (Stop Recording):** Press the STOP button (GPIO33) or allow the 60-second limit to expire.
- **Step 6 (WAV Reconstruction):** The serial bridge verifies CRC32 checksums, detects the terminal frame, and writes the WAV file.
- **Step 7 (Automated Analysis):** The bridge posts the WAV to /api/v1/session/<session_id>/analyze, triggering windowing and HuBERT-D inference.
- **Step 8 (Event Aggregation):** Overlapping positive windows are merged into detected disfluency events.
- **Step 9 (Database Persistence):** Session metadata, window probabilities, and aggregated events are stored in SQLite.
- **Step 10 (Session Review):** The dashboard automatically renders the session timeline, event metrics, and longitudinal trends.

10.6 Validation and Deployment

The deployment model operates entirely on-premise on the host workstation connected to the ESP32 capture unit via USB-UART (as shown in the Deployment Diagram, Fig. 8). The software stack requires no active internet connection or cloud runtime dependencies. Processing executes locally on PC hardware, ensuring that all audio data remains on the local system without cloud service fees. VoxFlow is a research prototype and is not a medical diagnostic system.

11. RESULTS AND DISCUSSION

11.1 Model Results

We evaluated three distinct model architectures on the speaker-exclusive test split (6,812 clips across 249 unseen speakers): (1) a traditional baseline utilizing 13 MFCCs

with Support Vector Machines (MFCC + SVM), (2) a fine-tuned wav2vec 2.0 transformer, and (3) our fine-tuned HuBERT-D model. As presented in Table 10, HuBERT-D achieved the highest overall performance with a Macro F1 score of 0.4599 and a Mean ROC-AUC of 0.8099, outperforming wav2vec 2.0 (0.4498 / 0.8020) and substantially surpassing the MFCC + SVM baseline (0.3121 / 0.6633).

Table 10. Model Evaluation Comparison on Speaker-Exclusive Test Split

Model Architecture	Input Features	Macro F1	Mean ROC-AUC	Repetition F1	Prolongation F1	Block F1
MFCC + SVM (Baseline)	13 MFCCs + delta + delta-delta	0.3121	0.6633	0.4102	0.3215	0.2046
wav2vec 2.0 (Fine-tuned)	Raw Audio (16 kHz)	0.4498	0.8020	0.5412	0.4730	0.3352
HuBERT-D (Proposed)	Raw Audio (16 kHz)	0.4599	0.8099	0.5565	0.4841	0.3393

A granular analysis of HuBERT-D per-class performance reveals that Repetition detection achieved the highest F1 score (0.5565), followed by Prolongation (0.4841), while Block detection remained the most challenging class (0.3393). This pattern aligns with acoustic characteristics: repetitions and prolongations exhibit salient acoustic and phonetic cues (rhythmic repetition and sustained formants), whereas blocks are characterized by silent postural fixations that are acoustically difficult to distinguish from natural conversational pauses.

11.2 Session Analysis Results

Session processing tests confirm that sliding-window analysis paired with temporal event aggregation correctly measures continuous speech sessions. Slicing a 10-second speech recording produces 8 windows; when simulated repetitions occur across windows 2, 3, and 4 (timestamps 1.0s to 6.0s), the temporal aggregator consolidates the 3 positive windows into exactly 1 unified detected event spanning from 1.0s to 6.0s with peak confidence. This verifies that event consolidation prevents artificial inflation of disfluency counts.

11.3 System Discussion

The experimental and operational results confirm that VoxFlow achieves its primary design goals: reliable digital capture via ESP32, error-checked transport via VXF1, robust self-supervised disfluency detection via HuBERT-D, and clean visual analytics via Streamlit. Operating entirely on local PC hardware without cloud dependencies guarantees data privacy and eliminates operational costs.

12. LIMITATIONS AND FUTURE SCOPE

12.1 Limitations

While VoxFlow demonstrates a successful end-to-end implementation, several limitations are acknowledged:

- **Block Detection Sensitivity:** Block detection achieved an F1 of 0.3393. Silent blocks remain difficult to separate from natural pauses without visual articulatory tracking or lexical language model context.
- **Acoustic Domain Mismatch:** HuBERT-D was fine-tuned on podcast and studio recordings (SEP-28k / FluencyBank). Although the INMP441 provides high-quality digital audio, acoustic domain adaptation may improve performance under background noise.
- **Single-User Local Deployment:** The prototype is designed for a single workstation without multi-user authentication or role-based access control.

12.2 Future Scope

Future development will focus on the following enhancements:

- **Multimodal Block Detection:** Integrate facial landmark and articulatory motion tracking via webcam to detect silent oral fixations, significantly boosting block detection accuracy.
- **ASR & Lexical Integration:** Incorporate automatic speech recognition (Whisper) to align acoustic disfluency predictions with text transcripts for syllable-accurate %SS calculation.
- **Therapy Usability Studies:** Conduct formal usability studies in collaboration with speech therapists to assess workflow utility in practical sessions.
- **Edge AI Inference:** Explore quantized on-device inference using ESP32-S3 or low-power edge accelerators for fully standalone pocket analyzers.

13. CONCLUSION

In this project, we designed, implemented, and verified VoxFlow, an end-to-end speech fluency and disfluency session analyzer. VoxFlow bridges the gap between theoretical machine learning clip classification and practical session assessment by uniting low-cost embedded hardware capture (ESP32 + INMP441), an error-checked serial protocol (VXF1 with CRC32 verification), self-supervised deep learning modeling

(HuBERT-D), algorithmic temporal event aggregation, relational SQLite persistence, and an interactive Streamlit visual analytics dashboard.

Evaluated under strict speaker-exclusive constraints across 519 distinct speakers, HuBERT-D achieved a Macro F1 score of 0.4599 and a Mean ROC-AUC of 0.8099 on unseen voices, outperforming wav2vec 2.0 (0.4498) and an MFCC + SVM baseline (0.3121). The temporal event aggregator successfully consolidates overlapping sliding-window detections into bounded detected events, eliminating redundant multi-counting. The entire software suite passes all 38 automated unit and integration tests. Operating entirely on local PC hardware without cloud runtime dependencies, VoxFlow provides an accessible, privacy-preserving, and scientifically grounded tool for automated speech fluency analysis. VoxFlow is a research prototype and is not a medical diagnostic system.

REFERENCES

- [1] S. A. Sheikh, M. Sahidullah, F. Hirsch and S. Ouni, "Machine learning for stuttering identification: Review, challenges and future directions," *Neurocomputing*, vol. 514, pp. 385–402, 2022, doi: 10.1016/j.neucom.2022.10.015.
- [2] C. Lea, V. Mitra, A. Joshi, S. Kajarekar and J. P. Bigham, "SEP-28k: A dataset for stuttering event detection from podcasts with people who stutter," in *Proc. IEEE ICASSP*, 2021, pp. 6798–6802, doi: 10.1109/ICASSP39728.2021.9413520.
- [3] N. Bernstein Ratner and B. MacWhinney, "Fluency Bank: A new resource for fluency research and practice," *Journal of Fluency Disorders*, vol. 56, pp. 69–80, 2018, doi: 10.1016/j.jfludis.2018.03.002.
- [4] P. Howell, S. Davis and J. Bartrip, "The University College London Archive of Stuttered Speech (UCLASS)," *Journal of Speech, Language, and Hearing Research*, vol. 52, no. 2, pp. 556–569, 2009, doi: 10.1044/1092-4388(2009/07-0129).
- [5] T. Kourkounakis, A. Hajavi and A. Etemad, "FluentNet: End-to-end detection of stuttered speech disfluencies with deep learning," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 2986–2999, 2021, doi: 10.1109/TASLP.2021.3110146.
- [6] S. A. Sheikh, M. Sahidullah, F. Hirsch and S. Ouni, "StutterNet: Stuttering detection using time delay neural network," in *Proc. 29th European Signal Processing Conference (EUSIPCO)*, 2021, pp. 426–430, doi: 10.23919/EUSIPCO54536.2021.9616063.
- [7] S. A. Sheikh, M. Sahidullah, F. Hirsch and S. Ouni, "Advancing stuttering detection via data augmentation, class-balanced loss and multi-contextual deep learning," *IEEE Journal of Biomedical and Health Informatics*, vol. 27, no. 5, pp. 2553–2564, 2023, doi: 10.1109/JBHI.2023.3248281.
- [8] S. P. Bayerl, A. Wolff von Gudenberg, F. Hönig, E. Nöth and K. Riedhammer, "KSoF: The Kassel State of Fluency dataset – A therapy centered dataset of stuttering," in *Proc. 13th Language Resources and Evaluation Conference (LREC)*, 2022, pp. 1780–1787. [Online]. Available: <https://aclanthology.org/2022.lrec-1.189/>
- [9] S. P. Bayerl, D. Wagner, E. Nöth and K. Riedhammer, "Detecting dysfluencies in stuttering therapy using wav2vec 2.0," in *Proc. Interspeech*, 2022, pp. 2868–2872, doi: 10.21437/Interspeech.2022-10908.
- [10] S. P. Bayerl, D. Wagner, E. Nöth, T. Bocklet and K. Riedhammer, "The influence of dataset partitioning on dysfluency detection systems," in *Text, Speech, and Dialogue (TSD 2022)*, *Lecture Notes in Computer Science*, Springer, 2022, pp. 423–436, doi: 10.1007/978-3-031-16270-1_35.
- [11] S. P. Bayerl, M. Gerczuk, A. Batliner, C. Bergler, S. Amiriparian, B. Schuller, E. Nöth and K. Riedhammer, "Classification of stuttering – The ComParE challenge and beyond," *Computer Speech & Language*, vol. 81, art. 101519, 2023, doi: 10.1016/j.csl.2023.101519.
- [12] A. Baevski, Y. Zhou, A. Mohamed and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020, pp. 12449–12460.
- [13] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov and A. Mohamed, "HuBERT: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3451–3460, 2021, doi: 10.1109/TASLP.2021.3122291.
- [14] VoxFlow Project Team, "VoxFlow — Speech Fluency & Disfluency Session Analyzer," *GitHub Repository*, Oct. 2026. [Online]. Available: <https://github.com/PetaSivaNandhanReddy/voxflow>
- [15] InvenSense (TDK), "INMP441: Omnidirectional Microphone with Bottom Port and I2S Digital Output," *Product Data Sheet*, Rev. 1.1.

- [16] Espressif Systems, "ESP32 Series Datasheet & Technical Reference Manual," Espressif Systems, 2024. [Online]. Available: <https://www.espressif.com/>
- [17] Meta AI, "facebook/hubert-base-ls960 Model Card," Hugging Face, 2021. [Online]. Available: <https://huggingface.co/facebook/hubert-base-ls960>

APPENDICES

Appendix A — Key Configuration Parameters

Table 11. Key System Configuration Parameters

Configuration Setting	Configured Value	Defined Location / Module
Sampling Rate / Audio Format	16,000 Hz, 16-bit signed PCM, Mono	Firmware (I2S), configs/config.py
I2S Frame Size / FreeRTOS Queue	1,024 samples (2,048 bytes) / 16 slots	Firmware (voxflow_esp32_firmware.ino)
Session Duration Limits	Min 3.0 s, Max 60.0 s (960,000 samples)	Firmware, Serial Bridge, Session Engine
Serial Baud Rate & Protocol	460,800 baud, VXF1 framed binary	Firmware, hardware/bridge/serial_bridge.py
Sliding Window / Hop Size	3.0 s (48,000 samples) / 1.0 s (16,000 samples)	configs/config.py, Session Engine
Decision Thresholds	Repetition 0.77, Prolongation 0.79, Block 0.57	configs/config.py, models/Hubert_D/val_summary.json
Temporal Event Merge Gap	1.5 seconds (maximum cluster window gap)	app/services/event_aggregator.py
RMS Silence Threshold	RMS < 0.0001 (triggers HTTP 422)	app/services/session_engine.py
Database File Path	app/voxflow.db (SQLite relational schema)	configs/config.py, DatabaseManager
Microservice Network Ports	Flask Backend: 5000, Streamlit UI: 8501	configs/config.py, Streamlit default

Appendix B — Testing and Validation Evidence Status

All 38 automated unit and integration tests execute and pass (38/38 passed). The complete test execution logs, API response payloads (confirming HTTP 422 rejection on 2-second and silent audio, and HTTP 200 success on 5-second and 60-second audio), and UI screenshots are documented in the companion Testing & Validation Evidence document.

Hardware implementation assets (pin wiring, button triggers, LED indication, and 60-second limit logic) are implemented in the firmware and circuit schematic. Formal physical evidence (oscilloscope traces, physical button press logs, optical LED recordings, and hardware audio session logs) was not recorded in the repository.

Appendix C — Supporting Documents and Repository Assets

The complete VoxFlow engineering artifacts—including ESP32 C++ firmware, Python serial bridge, Flask backend, HuBERT-D inference engine, Streamlit dashboard, automated test suites, and PlantUML diagram source models—are maintained in the official repository: <https://github.com/PetaSivaNandhanReddy/voxflow>.